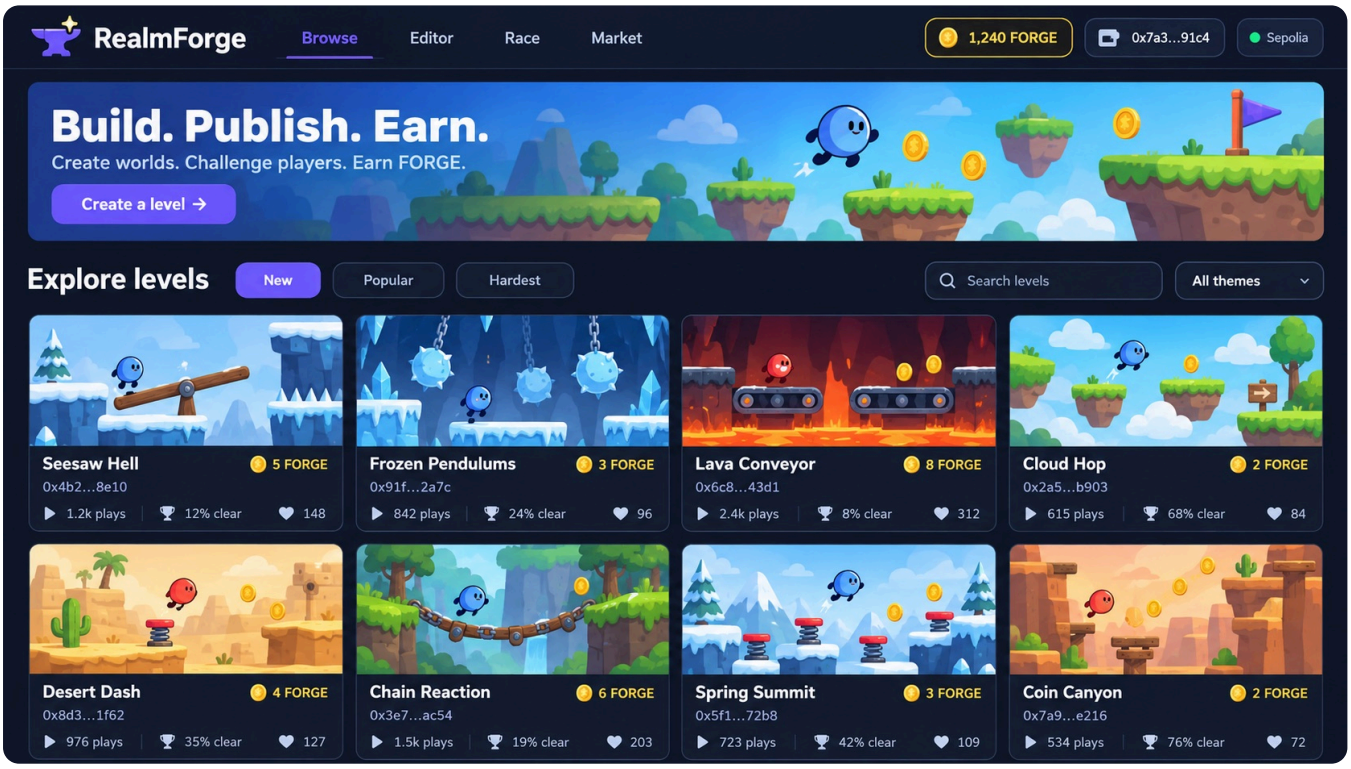
RealmForge

Technical Specification — Web Mario-Maker-style platformer with on-chain levels (ERC-721) and economy (ERC-20)

Module Blockchain BK-01 · Team of two · October 2026



RealmForge — Technical Specification

A web-based "Mario Maker"-style platformer: players build levels with rich physics, publish them as NFTs, and earn tokens when others play.

Module: Blockchain BK-01 (team of two) · Target: browser DApp · Chain: Ethereum testnet (Sepolia or Base Sepolia)

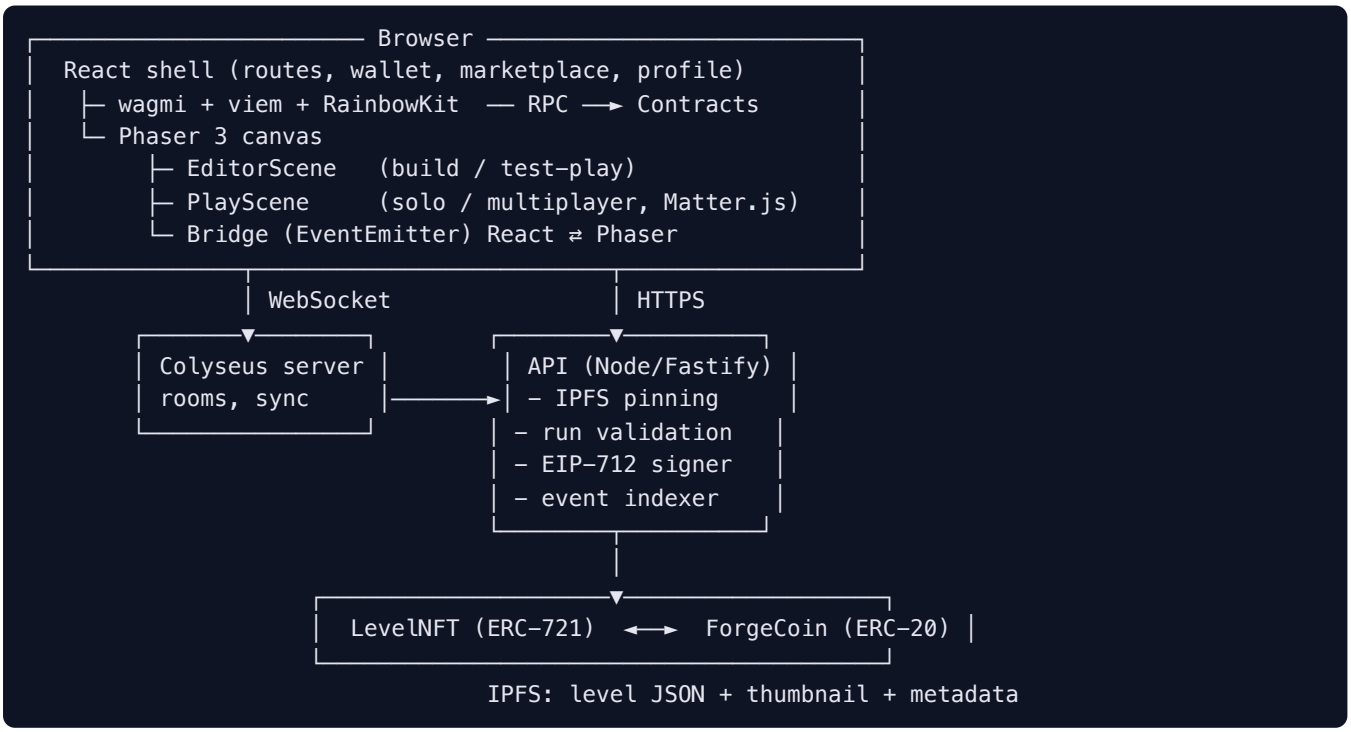
1. Scope and principles

Principle	Decision
Platform	**Web only** (desktop and mobile browsers). No native build.
Engine	**Phaser 3 + Matter.js physics**, TypeScript, Vite, React UI shell
Blockchain role	Ownership (levels), economy (token), settlement (rewards, fees). **No gameplay on-chain.**
Contracts	2 Solidity contracts, **one student per contract**: one NFT, one fungible token
Reuse	Use public, permissively licensed assets and code wherever possible (see §8)
IP	**No Nintendo assets or names.** All art is CC0 (Kenney). "Mario Maker" is only the gameplay reference.

Out of scope (MVP)

On-chain physics or level data, a real-money mainnet deployment, native desktop builds, and voice or text chat.

2. System architecture



Trust model: The game server is the authority that signs run results. Contracts check the signature, a nonce, a deadline, the chain ID, and the contract address, using EIP-712. This is a deliberate trade-off. Explain it at the oral exam (AA4/AA6).

---

3. Game specification

3.1 Physics (Matter.js)

- **Player controller:** compound Matter body (main body plus ground, left, and right sensors). Supports coyote time (100 ms), jump buffer (100 ms), variable jump height, wall slide, and wall jump.
- **Fixed timestep** at 60 Hz (`matter.runner.isFixed = true`) so replays and validation stay deterministic enough.
- **Collision categories:** player, enemy, solid, one-way, hazard, pickup, sensor.

3.2 Level elements (editor palette)

Category	Elements	Physics
Terrain	ground, brick (breakable), question block, ice (low friction), slime (high friction), one-way platform, semi-solid	static bodies, per-tile friction
Hazards	spikes, lava, saw (rotating), falling block	sensor or kinematic
Dynamic	crate (pushable), seesaw (hinge constraint), pendulum or wrecking ball, chain bridge (constraint chain), spring or trampoline, conveyor belt	dynamic bodies and constraints
Moving	moving platform (path with waypoints), elevator, rotating platform	kinematic, tweens
Enemies	walker, shell (kickable projectile), flyer (sine path), jumper, cannon	dynamic with AI state machine
Items	coin, big coin, key, power-up (double jump, dash)	sensor
Flow	start, checkpoint, goal, door or pipe pairs (teleport), key-lock door, switch (toggles blocks)	sensor

Each element has editable properties in the property panel: speed, direction, path, link target, and timer.

3.3 Level editor

- Grid placement (18 px or 21 px Kenney tiles) for a level of up to **300 × 30 tiles**.
- Tools: place, erase, select or drag, rotate, property panel, and link mode (door↔door, switch→blocks).
- Undo and redo (command pattern, 100 steps), copy and paste of a selection.
- **Test-play toggle** (key `T`) switches instantly between edit and play from the cursor position.
- Themes: grass, desert, snow, castle (Kenney tilesets). Background music is chosen per level.
- Autosave to `localStorage`, plus JSON export and import.
- **Publish flow:**
- Clear check: the creator must finish the level. The server validates the run and signs a `clearProof`.
- Generate a thumbnail with `game.renderer.snapshot`.
- Pin the level JSON, thumbnail, and metadata to IPFS.
- Call `LevelNFT.publish(...)`.

3.4 Level file format (`level.v1.json`, stored on IPFS)

```
{
  "schema": 1,
  "meta": { "name": "Seesaw Hell", "theme": "snow", "timeLimit": 300, "music": "snow_01" },
  "size": { "w": 220, "h": 30, "tile": 18 },
  "layers": { "terrain": "<RLE-encoded uint8 grid>" },
  "entities": [
    { "id": 1, "type": "seesaw", "x": 40, "y": 22, "props": { "length": 6 } },
    { "id": 2, "type": "door", "x": 80, "y": 20, "props": { "link": 3 } }
  ],
  "spawn": { "x": 2, "y": 25 }, "goal": { "x": 215, "y": 24 }
}
```

On-chain we store `layoutHash = keccak256(canonicalJSONBytes)` so a pinned file can always be checked against its NFT.

3.5 Game modes

Mode	Description	Networking
Solo	Play any published level, time trial	Inputs recorded and sent to the server at the end
Co-op (2–4)	Same level, shared checkpoints, everyone must reach the goal	Colyseus room, 20 Hz state patches
Versus race (2–4)	First to the goal wins the room pot	Colyseus room, server timing
Ghost race (async)	Race against the top-3 replays of a level	Replays from the API

3.6 Multiplayer and anti-cheat

- The Colyseus room owns room state: players, checkpoints, timers, and finish order.
- Clients simulate their own player locally (client prediction). The server checks plausibility: maximum speed per tick, checkpoint order, and minimum possible time per level (computed at publish).
- Solo runs send a compressed input log (`frame, buttons` bitmask). Re-simulating the log headlessly on the server is a **stretch goal**. Explain its limits at the oral exam.
- When a run passes, the server signs the EIP-712 `RunResult` (§5.2).

3.7 Web pages (React)

`/` home · `/browse` level browser (filters: new, popular, hardest, by creator) · `/level/:id` details, leaderboard, play, buy · `/editor` · `/play/:id` · `/room/:code` multiplayer lobby · `/profile/:address` created levels, owned levels, coin balance, clears · `/market` NFT listings.

3.8 Interface mockups

AI-generated concept mockups (Codex image generation). The final game art will use Kenney CC0 assets.

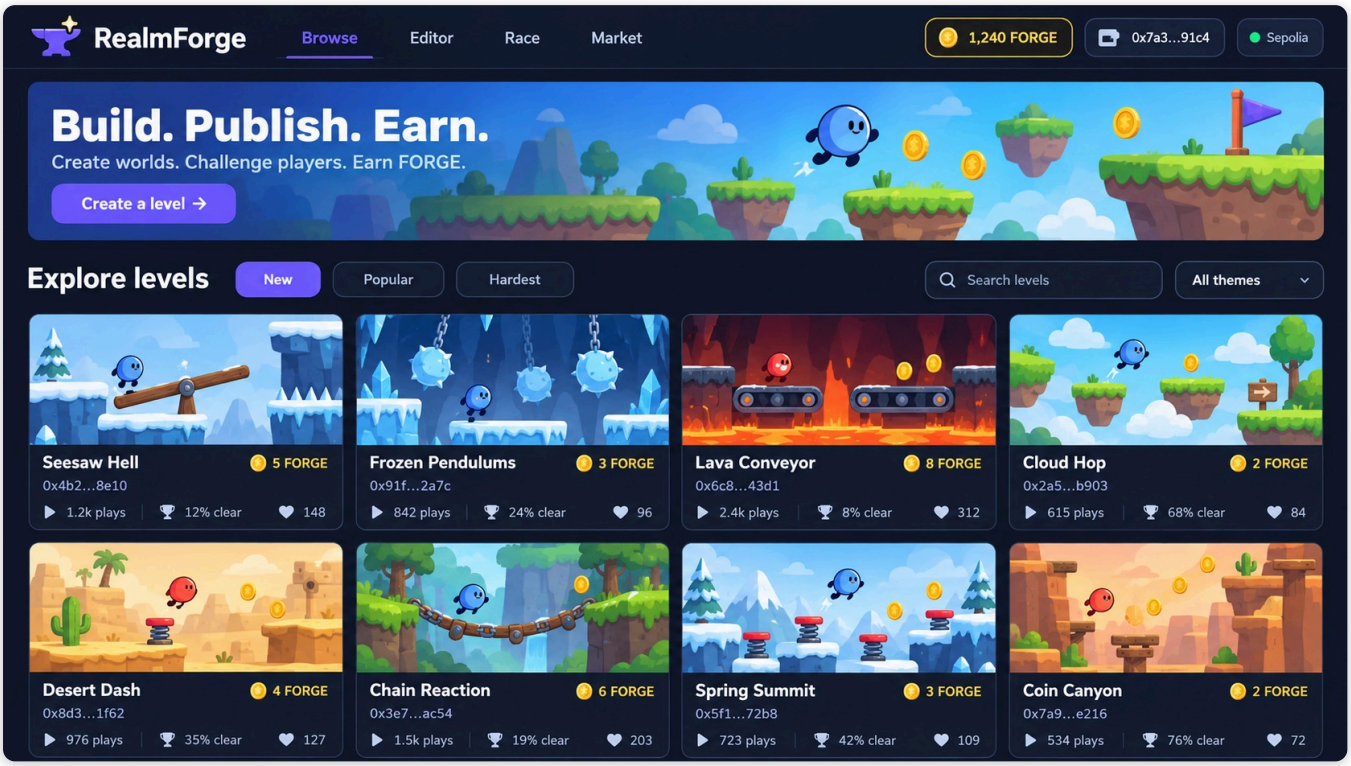


Figure 1: Home and level browser (/browse): filters, play fee, plays, clear rate, likes

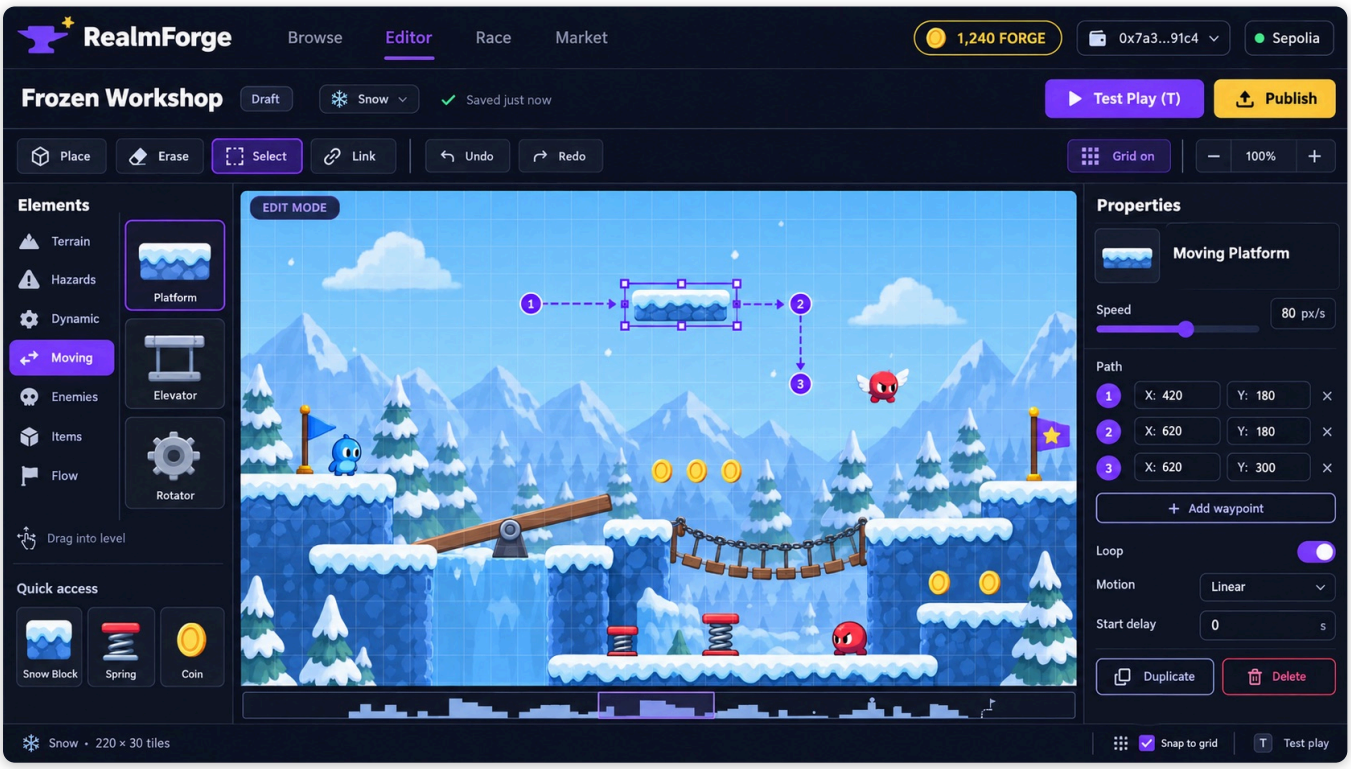


Figure 2: Level editor (/editor): element palette, physics elements (seesaw, chain bridge, springs), moving-platform waypoints, property panel, Test Play

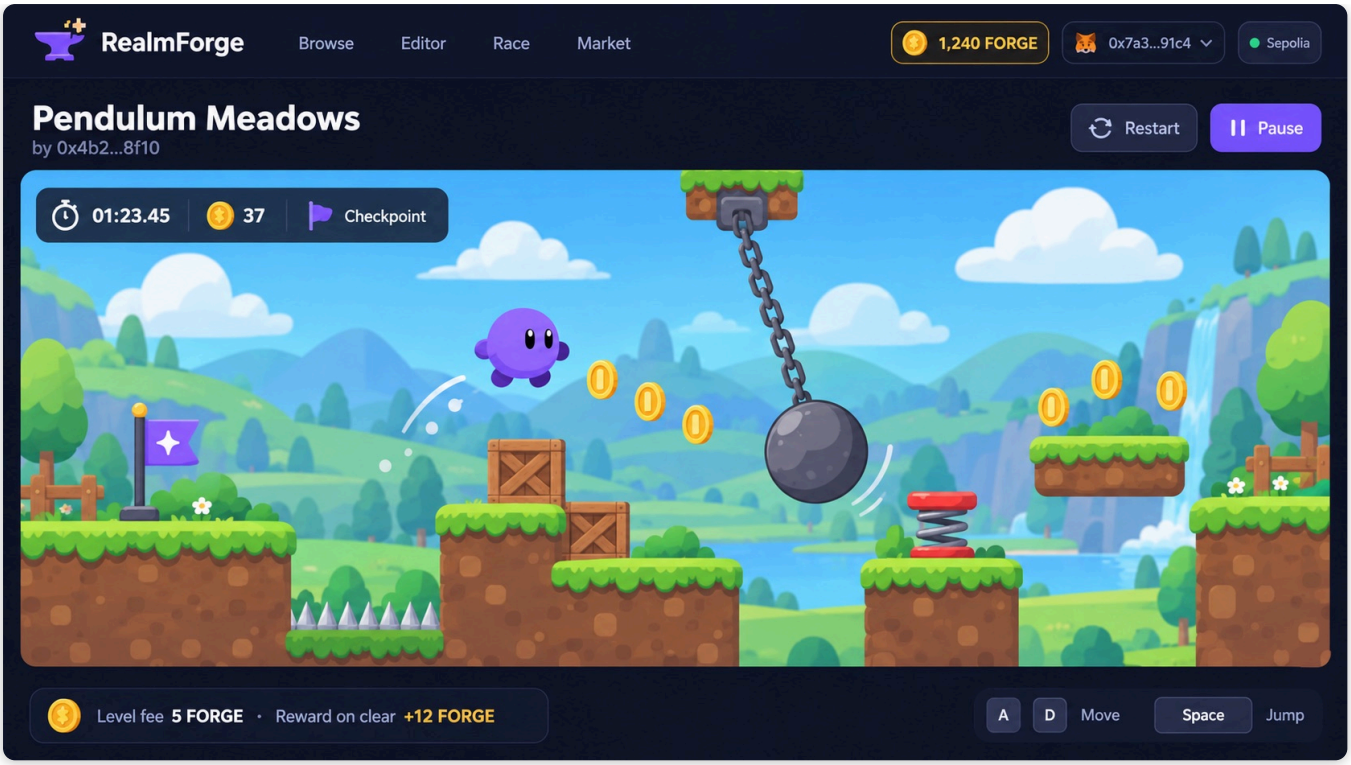


Figure 3: Solo play (/play/:id): Matter.js pendulum, crates, springs; HUD with timer, coins, checkpoint; level fee and reward

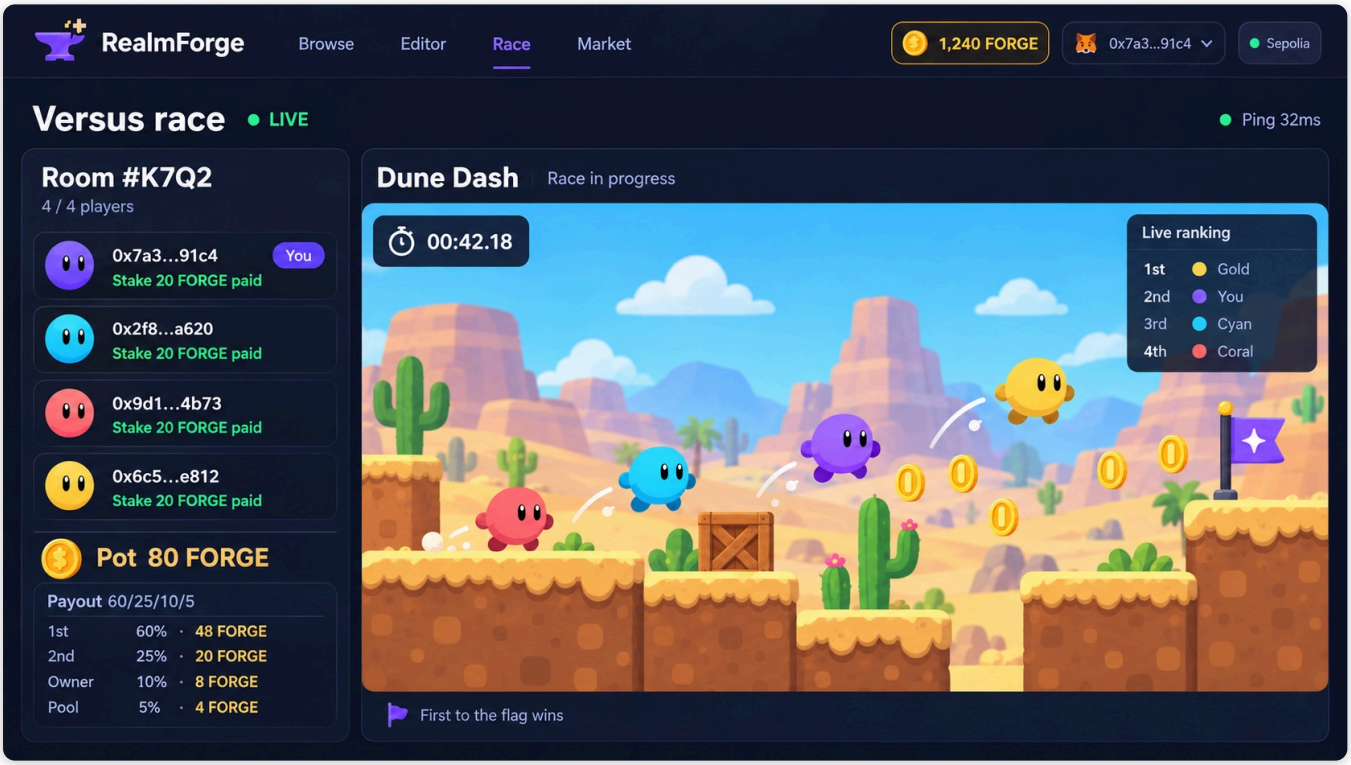


Figure 4: Versus race (/room/:code): ForgeCoin stake escrow, 80 FORGE pot, 60/25/10/5 payout, live ranking

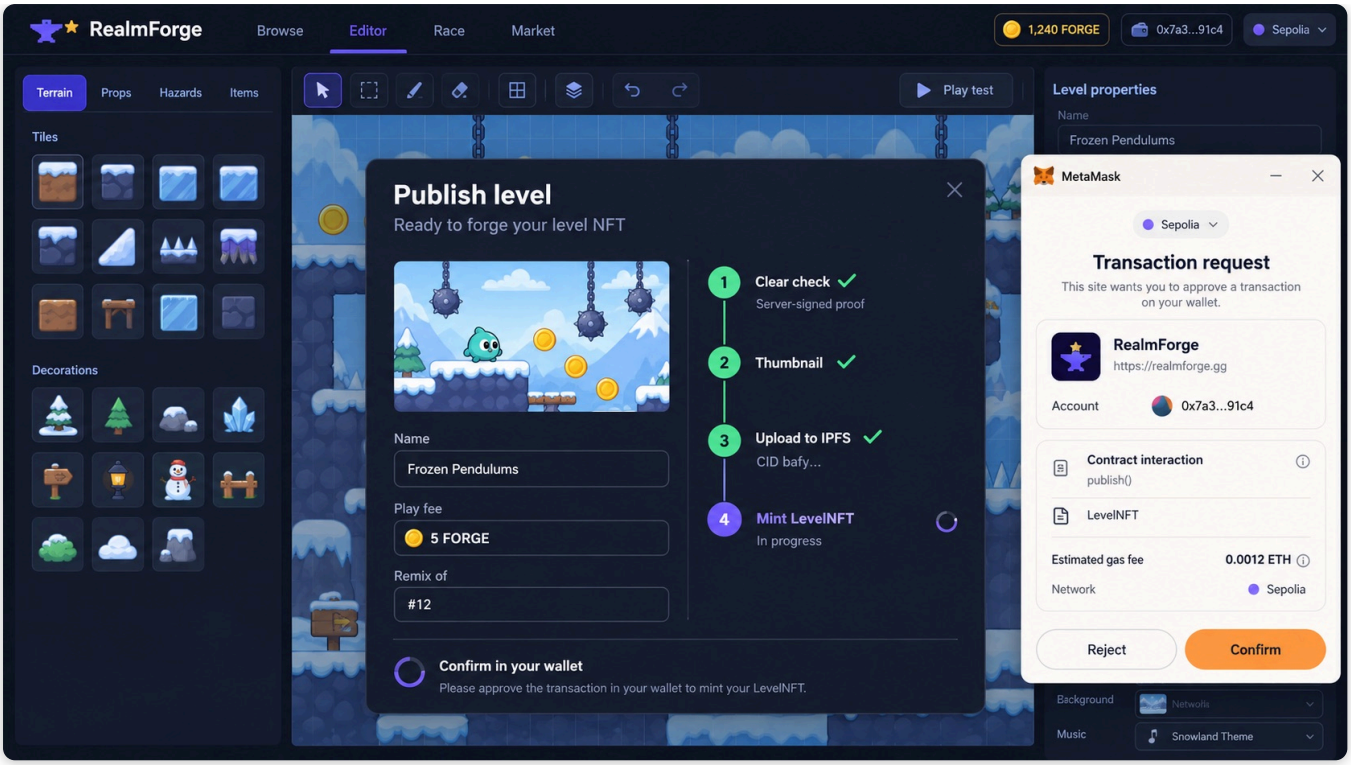


Figure 5: Publish flow: clear check, thumbnail, IPFS upload, LevelNFT.publish() wallet confirmation on Sepolia



Figure 6: Level detail and market (/level/:id): ERC-721 ownership, 5% royalty, on-chain leaderboard, creator earnings, fixed-price listing

4. Contract A — `LevelNFT.sol` (Student A) — ERC-721

Purpose: Each published level is an NFT. The owner receives play fees, can sell the level, and remixes pay royalties to the original creator.

Base (OpenZeppelin v5): `ERC721`, `ERC721URIStorage`, `ERC2981` (royalties), `AccessControl`, `ReentrancyGuard`, `EIP712`.

4.1 Storage

```
struct Level {
    address creator;       // original author (immutable)
    bytes32 layoutHash;    // keccak256 of canonical level JSON
    uint256 parentId;      // 0 = original, else remix source
    uint96  playFee;       // in FORGE (ERC-20 units), set by owner
    uint32  plays;
    uint32  clears;
    uint32  likes;
    uint32  bestTimeMs;
    address bestPlayer;
    bool    active;
}
mapping(uint256 => Level) public levels;
mapping(bytes32 => bool) public hashUsed;        // no duplicate publishing
mapping(uint256 => uint256) public listingPrice; // fixed-price market (in ETH)
mapping(uint256 => mapping(address => bool)) public liked;
mapping(bytes32 => bool) public usedProof;       // clear-proof replay protection
uint256 public nextId = 1;

bytes32 public constant GAME_ROLE   = keccak256("GAME_ROLE");   // granted to ForgeCoin
bytes32 public constant SIGNER_ROLE = keccak256("SIGNER_ROLE"); // game server key
uint96  public constant MAX_PLAY_FEE = 100e18;
uint96  public constant REMIX_ROYALTY_BPS = 500;  // 5% on secondary sales
```

4.2 Functions

Function	Access	Behavior
`publish(bytes32 layoutHash, string uri, uint96 playFee, uint256 parentId, uint256 deadline, bytes clearSig)`	anyone	Checks that `hashUsed` is false, the fee is at most the max, the parent exists if a remix, and the **clear proof is signed by SIGNER_ROLE** (EIP-712 `ClearProof{creator, layoutHash, deadline}`). Mints to the sender, sets royalty (ERC-2981) for the creator, and emits `LevelPublished`.
`setPlayFee(id, fee)`	token owner	Updates the fee (capped).
`setActive(id, bool)`	token owner	Takes the level offline or brings it back.
`like(id)`	anyone, once per level	`likes++`
`list(id, priceWei)` / `unlist(id)`	token owner	Fixed-price listing.
`buy(id)`	anyone, payable	Pays the royalty to the creator (and to the parent creator if a remix), then the rest to the seller. Uses the **pull pattern** (`pendingWithdrawals`), then transfers the NFT and clears the listing.
`withdraw()`	anyone	Pulls pending ETH. `nonReentrant`.

Function	Access	Behavior
`recordRun(id, player, timeMs, cleared)`	**GAME_ROLE only** (ForgeCoin)	`plays++`, `clears++` if cleared, and updates best time and player.
`levelInfo(id)` view	—	Returns `(owner, creator, playFee, active)` for ForgeCoin.
`tokenURI`, `royaltyInfo`, `supportsInterface`	—	Standard.

4.3 Events

`LevelPublished(id, creator, layoutHash, parentId, playFee)` · `PlayFeeUpdated(id, fee)` · `LevelToggled(id, active)` · `Liked(id, user)` · `Listed(id, price)` · `Unlisted(id)` · `Sold(id, from, to, price)` · `RunRecorded(id, player, timeMs, cleared)` · `NewRecord(id, player, timeMs)`

4.4 Invariants and security (AA6)

- `layoutHash` is unique and `creator` never changes.
- An active listing is cleared on any transfer (override `_update`), which prevents stale listings.
- Only ForgeCoin can call `recordRun`, enforced by role.
- Pull payments plus `nonReentrant` on `buy` and `withdraw`.
- EIP-712 domain binds `chainId` and `address(this)`; `usedProof` blocks replays; `deadline` limits how long a proof is valid.
- Gas: no enumerable extension; the leaderboard history comes from events (indexer), not storage.

---

5. Contract B — `ForgeCoin.sol` (Student B) — ERC-20

Purpose: The in-game currency. Players pay level fees in FORGE, earn FORGE for clears and records, and creators earn from plays. Season prize pool.

Base (OpenZeppelin v5): `ERC20`, `ERC20Permit` (gasless approve), `ERC20Capped`, `AccessControl`, `ReentrancyGuard`, `EIP712`, `Pausable`.

5.1 Tokenomics

Parameter	Value
Cap	100,000,000 FORGE
Faucet (new wallet)	100 FORGE once per address
Play fee split	**70% level owner · 20% season pool · 10% treasury** (some of the fee can be burned instead of sent to treasury)
Clear reward (minted)	`baseReward (5) + difficultyBonus` with difficulty = `1 − clears/plays`, capped at 50
First clear of a level	+20 FORGE
New record	+10 FORGE
Daily mint cap per player	200 FORGE (anti-farming)
Season	30 days. Pool goes to the top 10 players and top 10 creators.

5.2 EIP-712 run result (signed by the game server)

```
struct RunResult {
    address player;
    uint256 levelId;
    uint32  timeMs;
    bool    cleared;
    uint16  coins;
    uint256 nonce;
    uint256 deadline;
}
```

5.3 Functions

Function	Access	Behavior
`claimFaucet()`	once per address	Mints 100 FORGE.
`settleRun(RunResult r, bytes sig)`	player (msg.sender == r.player)	Verifies the signature, nonce, and deadline. Reads `levelNFT.levelInfo` (must be active). Takes the play fee from the player and splits it per §5.1. Mints the reward (respecting the daily cap and the token cap). Calls `levelNFT.recordRun`. Emits `RunSettled`.
`settleRunWithPermit(RunResult r, bytes sig, uint256 v, bytes32 rr, bytes32 s, uint256 permitDeadline)`	player	Same as `settleRun`, with ERC-20 permit included, so no separate approve transaction.
`createRaceRoom(bytes32 roomId, uint256 levelId, uint256 stake)` / `joinRace(roomId)`	players	Stake is escrowed.
`settleRace(bytes32 roomId, address[] ranking, bytes sig)`	anyone, server-signed	Pays 1st 60%, 2nd 25%, owner 10%, pool 5%. Includes a refund path after a timeout.
`closeSeason(address[] players, address[] creators, uint256[] shares, bytes sig)`	DEFAULT_ADMIN	After `seasonEnd`: distributes the pool, `season++`.
`setSigner(addr)` / `pause()` / `unpause()` / `withdrawTreasury(to)`	admin	—

5.4 Events

`FaucetClaimed(user)` · `RunSettled(player, levelId, timeMs, cleared, reward, feePaid)` · `FeeSplit(levelId, owner, ownerAmt, poolAmt, treasuryAmt)` · `RaceCreated / RaceJoined / RaceSettled / RaceRefunded` · `SeasonClosed(season, pool)`

5.5 Invariants and security (AA6)

- `totalSupply <= cap`. Minted rewards never exceed the daily cap.
- Each nonce is used once, and the signature is bound to chain, contract, and deadline.
- Checks-effects-interactions: state is updated before the external `recordRun` call, and the function is `nonReentrant`.
- Race escrow: the sum of stakes equals the sum of payouts. A timeout refund means funds can never get stuck.
- `Pausable` serves as an emergency circuit breaker.
- Known limit to explain: **the server signer is a centralization point**. A compromised key could mint up to the daily caps. Mitigations: rotate the signer, rate limits, multisig admin.

---

6. Contract interaction (agree on this interface on day 1)

```
// Owned by Student A, consumed by Student B
interface ILevelNFT {
    function levelInfo(uint256 id) external view
        returns (address owner, address creator, uint96 playFee, bool active);
    function recordRun(uint256 id, address player, uint32 timeMs, bool cleared) external;
}
```

Deploy order: 1. `ForgeCoin(signer)` 2. `LevelNFT(signer)` 3. `forgeCoin.setLevelNFT(levelNFT)` 4. `levelNFT.grantRole(GAME_ROLE, forgeCoin)`

Until A's contract is ready, Student B tests against a `MockLevelNFT`.

7. Off-chain services

Service	Stack	Responsibility
Game server	Colyseus (Node, TypeScript)	Rooms, state sync, timing, finish order
API	Fastify + viem	IPFS pinning (Pinata or Filebase), run validation, EIP-712 signing (`signTypedData`), replay storage
Indexer	viem `watchContractEvent` → SQLite or Postgres	Level browser, leaderboards, history (keeps storage off-chain)
Signer key	`.env`, never committed	Shown in the oral exam as the trust boundary

8. Reused public assets and code (licenses verified on GitHub on 30/10/2026)

Assets — all CC0 (commercial use OK, no attribution required)

Asset	Source	Use
Kenney **New Platformer Pack** (400+ sprites: tiles, characters, enemies, HUD, sounds)	kenney.nl/assets/new-platformer-pack	Main tileset, characters, enemies
Kenney **Pixel Platformer** + **Pixel Platformer Blocks**	kenney.nl / OpenGameArt	Alternative pixel theme, editor icons
Kenney UI Pack, Interface Sounds, Music Jingles	kenney.nl	Editor UI, SFX
OpenGameArt CC0 music (filter by license)	opengameart.org	Background tracks

Code

Repo	License	★	Use
`phaserjs/phaser`	MIT	40.6k	Engine (Phaser 3.x; Phaser 4 is out, but most templates target v3)
`phaserjs/template-react-ts`	MIT	202	**Starting point**: React ⇄ Phaser bridge, Vite, TypeScript
`ourcade/sidescrolling-platformer-template-phaser3`	MIT	23	Matter.js side-scroller player and state machine (base for the controller)

Repo	License	★	Use
`yandeu/phaser3-typescript-platformer-example`	MIT	88	TypeScript platformer structure, tile handling
`mikewesthad/phaser-matter-collision-plugin`	MIT	99	Clean Matter collision callbacks
`rexrainbow/phaser3-rex-notes`	MIT	1.3k	UI plugins (grid, drag, scroll panels) for the editor
`colyseus/colyseus`	MIT	7.5k	Multiplayer rooms
`wevm/wagmi` (+ viem, RainbowKit)	MIT	6.8k	Wallet and contract hooks
`OpenZeppelin/openzeppelin-contracts` (v5)	MIT	27k	ERC-721, ERC-20, ERC-2981, EIP-712, AccessControl
OpenZeppelin Contracts Wizard	MIT	—	Generate the base contracts, then extend them
Hardhat 3 (Solidity + TypeScript tests) **or** `foundry-rs/foundry`	MIT / Apache-2.0	10.7k	Compile, test, deploy

Reference only (no license or GPL, so read for ideas but **do not copy code**): `pratishshr/mario-maker` (JS editor design, no license), `SuperTux/supertux` (GPL-3.0, mature in-game editor UX), `mmatyas/supermariowar` (multiplayer modes, no license), `PolyMarsDev/Mario-Maker-Deconstructed` (mechanic breakdowns).

9. Repository layout

```
realmforge/
├─ contracts/          # Hardhat 3 or Foundry
│  ├─ src/LevelNFT.sol          (Student A)
│  ├─ src/ForgeCoin.sol         (Student B)
│  ├─ src/interfaces/ILevelNFT.sol
│  ├─ test/                     (unit + fuzz + integration)
│  └─ script/Deploy.s.sol
├─ web/                # React + Phaser (from template-react-ts)
│  └─ src/game/{scenes,entities,editor,physics,net}
├─ server/             # Colyseus + Fastify API + signer + indexer
└─ docs/               # this spec, architecture diagram, gas report, threat model
```

10. Responsibilities split

	Student A — LevelNFT	Student B — ForgeCoin
Contract	`LevelNFT.sol` + tests (≥ 90% coverage)	`ForgeCoin.sol` + tests (≥ 90% coverage)
Signatures	`ClearProof` EIP-712	`RunResult`, race, and season EIP-712
Frontend	Editor publish flow, `/browse`, `/level/:id`, `/market`	Faucet, play settlement, leaderboards, race lobby, wallet balance
Server	IPFS pinning + clear validation	Run signer + Colyseus race settlement
Game (shared)	Editor scene, tiles, properties, undo	Play scene, player controller, enemies, multiplayer
Oral exam	NFT standard, royalties, market security	Tokenomics, signatures, escrow, reentrancy

11. Testing and evaluation (AA6)

- **Unit tests:** each function, each revert path (wrong signer, used nonce, expired deadline, fee above cap, inactive level, cap exceeded).
- **Fuzz tests:** fee split sums to the full fee; stake equals payout; supply stays at or under the cap.
- **Integration test:** publish → settleRun → recordRun → buy NFT → new owner receives the fees.
- **Static analysis:** Slither report, discuss findings.
- **Gas report:** `publish`, `settleRun`, `buy`, `settleRace`; compare L1 and L2 cost.
- **Game QA:** 60 FPS on a mid-range laptop and phone; level load time under 2 s.

---

12. Milestones (aligned to the module sessions)

Session	Deliverable
S3–S4	Repo set up, interfaces frozen, contracts v0 + tests on a local chain, Phaser template running with Kenney tiles
S5	Player controller (Matter), 15 elements, editor place/erase/test-play, deploy to testnet
S6 (intermediate version)	Publish flow end to end (IPFS + mint), solo play + `settleRun`, browse page
S6 → S7	Multiplayer race + escrow, marketplace, seasons, Slither + gas report
S7 (final)	Demo: build a level → publish → friend plays and pays → creator earns → race 1v1 → sell the NFT

Cuts if behind schedule (in order)

Season distribution → co-op mode → remix royalties → race escrow (keep solo only). **Never cut:** both contracts, the token flow between them, signature checks, tests.